

“Linux Process Creation, Execution and Termination System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

**JALA GEYA REDDY
KAMTAM PRIYANKA**

**2520090123
2520090001**

Under the Esteemed Guidance of

M. Raghupathi
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Linux Process Creation, Execution and Termination System”** is a record of Bonafide work of **JALLA GEYA REDDY - 2520090123** , **KAMTAM PRIYANKA – 2520090123** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

JALLA GEYA REDDY 2520090123

KAMTAM PRIYANKA 2520090001

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **“Linux Process Creation, Execution and Termination System”** is a bonafide work done and submitted by **JALLA GEYA REDDY - 2520090123 , KAMTAM PRIYANKA – 2520090123**, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. K. Anusha**, and my guide **M. Raghupathi** , Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the **Department for Computer Science Engineering** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

JALLA GEYA REDDY 2520090123

KAMTAM PRIYANKA 2520090001

Abstract

The project “Linux Process Management and Custom OS Terminal” is a C-based Linux application developed to demonstrate important Operating System concepts through practical execution. The system provides a menu-driven interface from which the user can execute demonstrations of process creation, program replacement, inter-process communication, process termination, zombie processes, orphan processes, and a combined execution of all demonstrations. The project uses Linux/POSIX system calls such as `fork()`, `execlp()`, `execvp()`, `waitpid()`, `pipe()`, `dup2()`, `read()`, `write()`, `sleep()`, `chdir()`, `getcwd()`, and process-status information available through `/proc`. These calls are organized into small modules so that process behavior can be observed directly from the terminal. In addition to the demonstrations, the project provides a custom terminal mode. The terminal supports common Linux commands and built-in commands such as `pwd`, `cd`, `help`, and `exit`. It also supports basic pipelines by connecting the output of one process to the input of another process.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		6–12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	Implementation		30– 45
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	Execution and Demonstration		62– 67
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	Conclusion		79– 83
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	References		84– 86
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER 1 – INTRODUCTION

1.1 Background of the Project

An operating system manages processes and system resources. Process management is fundamental because running programs are represented as processes and the operating system controls their creation, execution, synchronization, and termination.

This project presents these concepts through a single menu-driven C application. Instead of studying system calls only theoretically, the application creates real Linux processes and displays their identifiers and observable behavior.

1.2 Problem Statement

Concepts such as `fork()`, `exec()`, waiting, process termination, zombies, orphans, and pipes can be difficult to understand from notes alone. Students need a simple environment in which these operations can be executed and observed.

1.3 Scope of the Project

- Process creation using `fork()`.
- Program replacement using `exec` functions.
- Parent-child synchronization using `waitpid()`.
- Normal process termination and exit status.
- Zombie and orphan process demonstrations.
- IPC using anonymous pipes.
- A custom terminal with built-in commands.
- Basic command pipelines.

1.4 Objectives of the Project

1. Understand child-process creation.
2. Understand `exec`-based program replacement.
3. Observe parent-child synchronization.
4. Understand process exit status.
5. Observe zombie and orphan behavior.
6. Understand pipe-based IPC.
7. Implement a basic command interpreter.

1.5 Importance of the Application

The application converts abstract OS concepts into observable terminal output. PIDs, PPIDs, process states, exit statuses, and command execution can be seen directly, making it useful for laboratory demonstrations.

1.6 Target Users / Audience

- Operating Systems students.
- System Programming students.
- Students learning Linux system calls.
- Faculty demonstrating process concepts.

CHAPTER 2 – OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The project is based mainly on process management and inter-process communication. Linux assigns a PID to each process and maintains parent-child relationships.

- `fork()` – creates a child process.
- `exec` functions – replace the current process image.
- `waitpid()` – waits for and reaps a selected child.
- `exit()` – terminates a process with an exit status.
- `pipe()` – creates a unidirectional IPC channel.
- `dup2()` – redirects a standard file descriptor.
- `getpid()/getppid()` – obtain process identifiers.
- `/proc` – exposes kernel process information.

2.2 Existing Methods / Commands

Linux provides utilities such as `ps` and shell pipelines, but they do not present all selected concepts as one structured educational application. Normally, students must use several commands or separate programs.

2.3 Proposed Approach

The proposed approach combines the selected demonstrations into one menu-driven C application. Each module performs a specific OS operation and prints the information needed for observation. A custom terminal is also included for practical command execution.

2.4 Advantages of the Proposed Approach

- Single application for multiple demonstrations.
- Uses real Linux system calls.
- Displays PIDs and process relationships.
- Checks the actual zombie state through `/proc`.
- Provides practical shell and pipeline experience.

CHAPTER 3 – SYSTEM REQUIREMENTS

3.1 Hardware Requirements

- Laptop or desktop capable of running Ubuntu/WSL.
- At least 4 GB RAM recommended.
- Keyboard and display for terminal interaction.

3.2 Software Requirements

- Ubuntu Linux or Ubuntu through WSL.
- GCC compiler.
- GNU/Linux command-line utilities.
- A text editor.
- Git/GitHub for version control if required.

3.3 Operating System Environment – Ubuntu / WSL

The project is intended for Linux because the demonstrations depend on Linux/POSIX process APIs and the /proc interface. Ubuntu/WSL provides the required environment.

3.4 Development Tools and Compiler

The project is written in C and compiled using GCC.

```
gcc project.c -o project
./project
```

CHAPTER 4 – TECHNOLOGY AND TOOLS USED

4.1 C Programming

C was selected because it provides direct access to Linux/POSIX process and file-descriptor APIs.

4.2 GCC Compiler

GCC converts the source code into an executable Linux program.

4.3 Linux System Calls

API	Purpose	Project Usage
fork()	Creates a child.	All process demonstrations and terminal execution.
execlp()	Replaces process image.	Runs ls -l in fork/exec demo.
execvp()	Executes command array.	External commands in terminal.
waitpid()	Waits for child.	Synchronization and termination.
exit()	Terminates process.	Child termination/error handling.
pipe()	Creates IPC channel.	Pipe demo and terminal pipelines.
dup2()	Redirects descriptors.	Connects pipeline stdin/stdout.
getpid()/getppid()	Gets process IDs.	Displays process relationships.
read()/write()	Descriptor I/O.	Orphan demonstration synchronization.
chdir()/getcwd()	Directory operations.	cd and pwd.

4.4 Linux Commands and Utilities

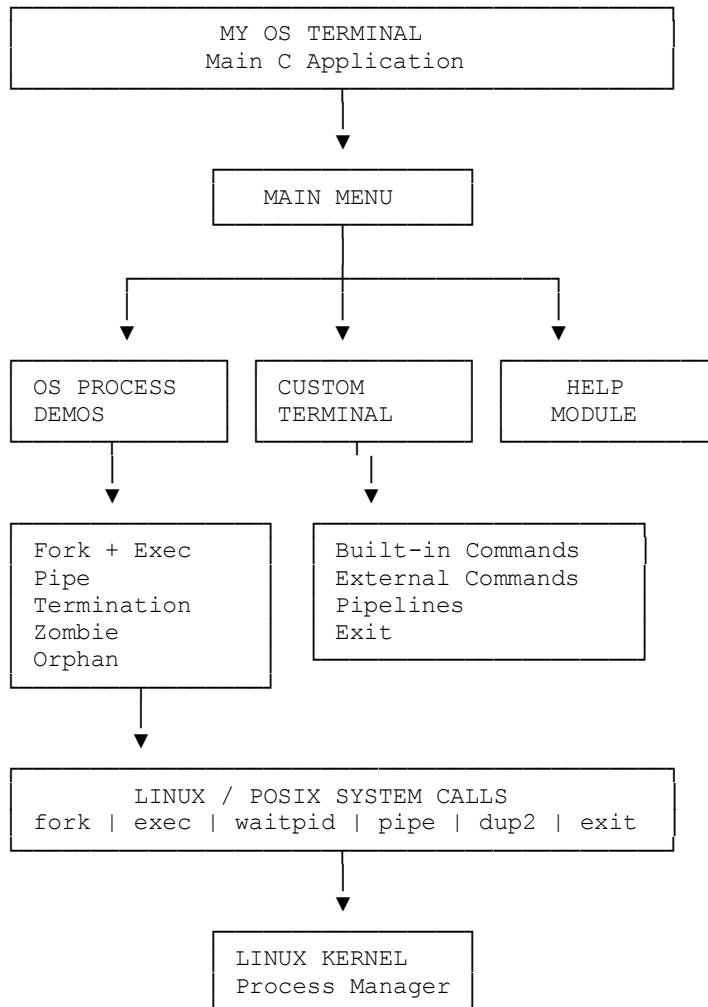
The custom terminal can execute external commands such as ls, ps, date, whoami, and cat. Built-ins include pwd, cd, help, and exit.

4.5 File System /proc Interface

The zombie demonstration opens /proc/<pid>/status and searches for the State field. This allows the program to display the actual Linux process state.

CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

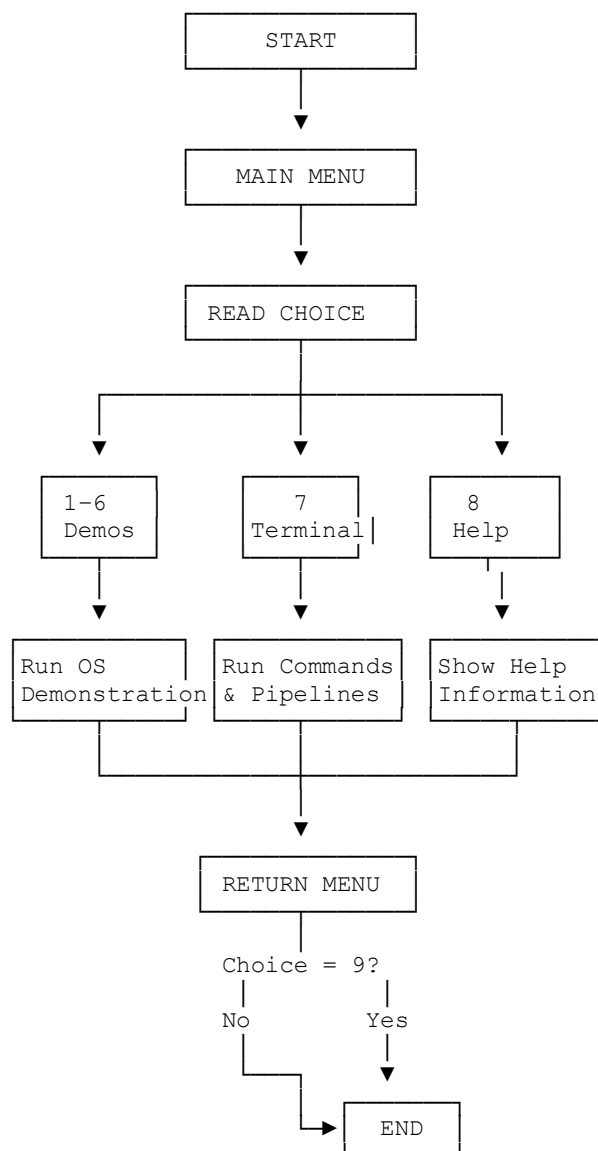
5.1 Overall System Architecture



5.2 System Workflow

8. Program starts in main().
9. show_menu() displays choices.
10. User selects an operation.
11. Selected module executes Linux calls.
12. Output is displayed.
13. Control returns to the menu; terminal mode remains active until exit is entered.

5.3 Flowchart / Data Flow



5.4 Module Description

Module	Description
Fork + Exec	Creates a child and executes <code>ls -l</code> using <code>execlp()</code> .
Pipe	Connects <code>ls</code> output to <code>wc -l</code> using pipe and <code>dup2</code> .
Termination	Child calls <code>exit(5)</code> ; parent reports exit status.
Zombie	Child exits; parent observes Z state through <code>/proc</code> , then calls <code>waitpid()</code> .
Orphan	Demonstration parent terminates while its child continues.
Run All	Runs all five demonstrations sequentially.
Normal Terminal	Runs built-ins, external commands, and basic pipelines.
Help	Displays available commands.

5.5 Input and Output Design

The main menu accepts an integer from 1 to 9. The custom terminal accepts text commands. Output includes headings, PIDs, PPIDs, process states, exit statuses, and command results

CHAPTER 6 – IMPLEMENTATION

6.1 Module-wise Implementation

6.1.1 Fork + Exec

fork() creates the child. The child prints its PID and PPID and calls execlp() to run ls -l. The parent waits with waitpid().

```
pid_t pid = fork();
if (pid == 0)
    execlp("ls", "ls", "-l", NULL);
else
    waitpid(pid, NULL, 0);
```

6.1.2 Pipe Demonstration

Two children are created. The first redirects stdout to the pipe and executes ls. The second redirects stdin from the pipe and executes wc -l.

```
pipe(pipefd);
pid1 = fork();          /* ls */
pid2 = fork();          /* wc -l */
dup2(pipefd[1], STDOUT_FILENO);
dup2(pipefd[0], STDIN_FILENO);
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
```

6.1.3 Process Termination

The child calls exit(5). The parent checks WIFEXITED(status) and obtains the value using WEXITSTATUS(status).

6.1.4 Zombie Process

The child exits while the parent delays waitpid(). The parent reads /proc/<pid>/status and observes State: Z (zombie). It then calls waitpid() and reports that the zombie has been removed.

6.1.5 Orphan Process

A demonstration parent creates another child and terminates. The child continues and prints its new PPID. A synchronization pipe informs the original terminal when the demonstration finishes.

6.1.6 Custom Terminal

The terminal uses fgets() to read commands, handles built-ins directly, detects the pipe symbol, and otherwise forks a child that uses execvp() for external commands.

6.2 Algorithm / Pseudocode

```
MAIN MENU
1. Start program.
2. Display menu.
3. Read choice.
4. Call the selected demonstration.
5. For option 7, start the custom terminal.
6. For option 8, display help.
```

7. For option 9, terminate.
8. Repeat.

CUSTOM TERMINAL

1. Display terminal instructions.
2. Read command.
3. If command is exit, return to menu.
4. If built-in, execute it.
5. If command contains '|', create a pipe and two children.
6. Otherwise fork and `execvp()` the command.
7. Wait for child processes.
8. Repeat.

6.3 Core C Implementation

The source is divided into functions including `show_process_state()`, `fork_exec_demo()`, `pipe_demo()`, `termination_demo()`, `zombie_demo()`, `orphan_demo()`, `os_demo()`, `show_help()`, `start_terminal()`, `show_menu()`, and `main()`.

6.4 Linux System Calls / APIs Used

The implementation uses process creation, execution, waiting, IPC, file-descriptor, directory, and process-status APIs. `perror()` is used for reporting system-call errors.

6.5 Process Implementation

After `fork()`, the return value distinguishes parent and child execution. Parents generally call `waitpid()` so children are synchronized and reaped.

6.6 IPC / Synchronization

Anonymous pipes transfer data between processes. The pipe demonstration connects `ls` to `wc -l`. The orphan demonstration uses a synchronization pipe to notify the original terminal.

6.7 Custom Terminal

The custom terminal supports `pwd`, `cd`, `help`, `exit`, external commands, and basic two-command pipelines. External commands execute in child processes while built-ins are handled by the terminal process.

CHAPTER 7 – EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

```
gcc project.c -o project
```

7.2 Execution Commands

```
./project
```

7.3 Sample Inputs and Outputs

7.3.1 Main Menu

```
1. Fork + Exec Demo
2. Pipe Demo
3. Process Termination Demo
4. Zombie Process Demo
5. Orphan Process Demo
6. Run All OS Demos
7. Start Normal Terminal
8. Help
9. Exit
Enter your choice:
```

7.3.2 Zombie Demonstration

```
Child process state:
State:  Z (zombie)

Now parent calls waitpid()...
Zombie removed.
```

7.3.3 Custom Terminal

```
Type 'help' to see available commands.
When you see 'myterminal$', type 'exit' to return to the menu.

myterminal$ exit
Returning to main menu...
```

7.3.4 Pipeline

```
myterminal$ ls | wc -l
3
```

7.4 Demonstration of OS Concepts

Concept	Module	Observation
Process creation	Fork + Exec	New child PID is displayed.
Program execution	Fork + Exec	Child runs ls -l.
IPC	Pipe	ls output is passed to wc -l.
Termination	Termination	Exit status 5 is reported.
Zombie	Zombie	State Z is observed before waitpid().
Orphan	Orphan	Child continues after demonstration parent exits.

Synchronization	waitpid()	Parent waits for child completion.
Shell	Normal Terminal	Commands execute through fork/exec.

Recommended screenshots for the final report are the main menu, each demonstration, the zombie state, the custom terminal, and the pipeline execution.

CHAPTER 8 – CONCLUSION

8.1 Summary of the Project

The project combines Linux process-management demonstrations and a simple custom terminal in one C application. The menu-driven design allows individual demonstrations or all demonstrations to be executed from one program.

8.2 Objectives Achieved

- Process creation with `fork()`.
- Program replacement with `exec` functions.
- Synchronization using `waitpid()`.
- Exit-status collection.
- Zombie and orphan demonstrations.
- Pipe-based IPC.
- Custom terminal and basic pipelines.

8.3 Results Achieved

The program executes in Ubuntu/WSL and produces observable process IDs, parent IDs, process states, exit statuses, and command results. The custom terminal provides a practical interface for command execution.

8.4 Skills and Learning Outcomes

- Linux process life cycle.
- `fork()` and `exec()`.
- `waitpid()` and synchronization.
- Zombie and orphan processes.
- `pipe()` and `dup2()`.
- File descriptors and redirection.
- Basic shell implementation in C.
- Compilation and testing in Ubuntu/WSL.

CHAPTER 9 – REFERENCES

9.1 Books

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, Operating System Concepts.
- W. Richard Stevens and Stephen A. Rago, Advanced Programming in the UNIX Environment.

9.2 Linux / GNU Documentation

- Linux manual pages for fork, exec, waitpid, pipe, dup2, exit, read, write, chdir, and getcwd.
- GNU Compiler Collection (GCC) documentation.

9.3 Online Resources

- Linux/POSIX system-call documentation used during implementation.
- Ubuntu documentation for Linux development and terminal usage.